Alma Mater Studiorum – University of Bologna

Master's Degree in Mechanical Engineering

Optimization and Machine Learning M

Bike Sharing Demand Prediction using Machine Learning Techniques

Professor: Andrea Lodi

Student: Federico Nanni

Academic Year 2025/2026

Contents

List of Figures

List of Tables

Listings

1 Theoretical Background

1.1 Machine Learning

Machine Learning concerns the development of computational methods capable of extracting useful information from data and using it to perform predictions or support decision-making processes.

A Machine Learning algorithm is generally governed by a set of parameters, denoted by $\boldsymbol{\Theta}$. The learning process consists of determining the values of these parameters that optimize a given objective function evaluated on a training dataset.

Let $\mathcal{D}_{\text{train}}$ denote the training dataset and let

$$f\left(\mathcal{D}_{\text{train}}, \boldsymbol{\Theta}\right) \tag{1.1}$$

be the objective function associated with the learning problem. The optimal parameter vector can be obtained either by maximizing a performance function,

$$\boldsymbol{\Theta}^* = \arg\max_{\boldsymbol{\Theta}} f\left(\mathcal{D}_{\text{train}}, \boldsymbol{\Theta}\right), \tag{1.2}$$

or, more commonly, by minimizing a loss function,

$$\boldsymbol{\Theta}^* = \arg\min_{\boldsymbol{\Theta}} f\left(\mathcal{D}_{\text{train}}, \boldsymbol{\Theta}\right). \tag{1.3}$$

The objective function can be optimized explicitly, for example through analytical or gradient-based methods, or implicitly through algorithms that iteratively modify the model parameters in order to improve their performance.

1.2 Supervised Learning

In supervised learning, the available dataset consists of observations for which both the input attributes and the corresponding desired output are known.

A supervised dataset can be represented as

$$\mathcal{D} = \left\{\left(\boldsymbol{x}^{(i)}, y^{(i)}\right)\right\}_{i=1}^{n}, \tag{1.4}$$

where:

- $\boldsymbol{x}^{(i)} \in \mathbb{R}^p$ is the vector of input attributes associated with observation i;
- $y^{(i)}$ is the corresponding target value;
- n is the number of available observations;
- p is the number of input features.

The objective is to determine a function

$$\hat{y} = h\left(\boldsymbol{x}; \boldsymbol{\Theta}\right) \tag{1.5}$$

capable of predicting the target value associated with a new input vector.

The information contained in the known input-output pairs is used during the training phase to estimate the model parameters. The trained model is then evaluated on observations that were not used during learning.

1.3 Regression Problems

Supervised learning problems can be divided into classification and regression problems.

In classification, the target variable belongs to a finite set of classes. In regression, instead, the target is a numerical quantity and the purpose of the model is to estimate its value as a function of the input attributes.

A regression model can therefore be expressed as

$$\hat{y} = h\left(\boldsymbol{x}; \boldsymbol{\Theta}\right), \qquad \hat{y} \in \mathbb{R}. \tag{1.6}$$

The difference between the observed target value and the corresponding prediction is defined as the residual:

$$e^{(i)} = y^{(i)} - \hat{y}^{(i)}. \tag{1.7}$$

The learning procedure aims to determine a model capable of producing sufficiently small residuals not only on the observations used for training, but also on previously unseen data.

In the present project, the target variable is `cnt`, which represents the total hourly number of bike rentals. Since this variable is numerical, the prediction task is formulated as a supervised regression problem.

1.4 Training and Testing Data

The performance of a Machine Learning model cannot be assessed only on the observations used during the learning process. A model could reproduce the training data accurately while providing poor predictions on new observations.

For this reason, the available dataset is divided into two disjoint subsets:

$$\mathcal{D} = \mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{test}}, \tag{1.8}$$

with

$$\mathcal{D}_{\text{train}} \cap \mathcal{D}_{\text{test}} = \varnothing. \tag{1.9}$$

The training set is used to estimate the model parameters, whereas the test set is used only after training to evaluate the predictive performance of the model on unseen data.

In this project, 80% of the available observations are assigned to the training set and the remaining 20% to the test set.

1.5 Model Parameters and Hyperparameters

It is important to distinguish between model parameters and hyperparameters.

Model parameters are determined during the learning process by optimizing the objective function. Hyperparameters, instead, control the structure or the behaviour of the learning algorithm and must be selected before model training.

Let $\boldsymbol{\Theta}$ denote the model parameters and let $\boldsymbol{H}$ denote the hyperparameters. For a fixed hyperparameter configuration $\boldsymbol{H}$, the learning process determines

$$\boldsymbol{\Theta}^*(\boldsymbol{H}) = \arg\min_{\boldsymbol{\Theta}} f\left(\mathcal{D}_{\text{train}}, \boldsymbol{\Theta}; \boldsymbol{H}\right). \tag{1.10}$$

Different reasonable values of $\boldsymbol{H}$ can be tested, and the configuration providing the best validation performance is selected.

For the Random Forest model considered in this project, relevant hyperparameters include:

- the number of trees;
- the maximum tree depth;
- the minimum number of samples required to split a node.

1.6 Performance Measures for Regression

The predictive performance of a regression model can be quantified by comparing the observed target values with the model predictions.

Let n_{test} be the number of observations in the test set. The Root Mean Squared Error is defined as

$$\text{RMSE} = \sqrt{\frac{1}{n_{\text{test}}} \sum_{i=1}^{n_{\text{test}}} \left(y^{(i)} - \hat{y}^{(i)}\right)^2}. \tag{1.11}$$

The RMSE penalizes large prediction errors because the residuals are squared. Lower values indicate better predictive performance.

A second metric is the Mean Absolute Error:

$$\text{MAE} = \frac{1}{n_{\text{test}}} \sum_{i=1}^{n_{\text{test}}} \left|y^{(i)} - \hat{y}^{(i)}\right|. \tag{1.12}$$

The MAE represents the average absolute prediction error and is expressed in the same unit as the target variable.

The coefficient of determination is defined as

$$R^2 = 1 - \frac{\displaystyle\sum_{i=1}^{n_{\text{test}}} \left(y^{(i)} - \hat{y}^{(i)}\right)^2}{\displaystyle\sum_{i=1}^{n_{\text{test}}} \left(y^{(i)} - \bar{y}\right)^2}, \tag{1.13}$$

where $\bar{y}$ is the average observed target value in the test set. Values of R^2 close to one indicate that the model explains a large fraction of the variability of the target.

2 Dataset Description and Preliminary Exploration

2.1 Dataset Structure

In Data Mining and Machine Learning applications, data can be represented as a collection of objects described by a set of attributes. An object may also be referred to as a record, instance, example or observation, whereas an attribute may be referred to as a variable, field or feature.

According to this representation, the dataset considered in this project is composed of hourly observations of a bike-sharing system. Each row represents one hour of operation, while each column describes a temporal, meteorological or demand-related characteristic associated with that observation.

The dataset contains

$$n = 17\,379 \tag{2.1}$$

observations and 20 original attributes. The data refer to the Capital Bikeshare system in Washington D.C. and cover the years 2011 and 2012. Rental information is aggregated on an hourly basis and combined with temporal and meteorological information.

The target variable is `cnt`, which represents the total number of bicycles rented during a given hour. The remaining variables initially constitute the set of candidate input attributes.

2.2 Attribute Classification

The preliminary analysis requires understanding the nature of each attribute, since different data types may require different preprocessing operations.

The attributes can first be distinguished into binary, discrete and continuous variables.

A binary attribute can assume only two values, usually encoded as 0 and 1. A discrete attribute assumes values belonging to a finite or countable set, whereas a continuous attribute assumes real values within a given interval.

In the present dataset, the attributes can be grouped as follows:

- **Binary attributes:** `yr`, `holiday`, `weekend` and `workingday`;
- **Discrete temporal attributes:** `mnth`, `hr` and `weekday`;
- **Nominal categorical attributes:** `season` and `weathersit`;
- **Continuous meteorological attributes:** `temp`, `atemp`, `hum`, `windspeed`, `windspeed_mph` and `comfindex`;
- **Count attributes:** `casual`, `registered` and `cnt`.

Table 2.1: Original attributes of the bike-sharing dataset.

Attribute	Type	Description
instant	Discrete	Record identifier
dteday	Date	Calendar date
season	Categorical	Season of the year
yr	Binary	Year: 0 for 2011, 1 for 2012
mnth	Discrete cyclic	Month, from 1 to 12
hr	Discrete cyclic	Hour, from 0 to 23
holiday	Binary	Holiday indicator
weekday	Discrete cyclic	Day of the week
weekend	Binary	Weekend indicator
workingday	Binary	Working-day indicator
weathersit	Categorical	Weather condition
temp	Continuous	Normalized temperature
atemp	Continuous	Normalized apparent temperature
hum	Continuous	Normalized humidity
windspeed	Continuous	Normalized wind speed
windspeed_mph	Continuous	Wind speed in miles per hour
comfindex	Continuous	Outdoor comfort index
casual	Count	Number of casual users
registered	Count	Number of registered users
cnt	Count	Total number of rentals

2.3 Preliminary Data Exploration

A preliminary data analysis was performed before applying any preprocessing operation. According to the data management methodology, this phase is aimed at identifying the main characteristics of the dataset and selecting the most appropriate tools for cleaning and subsequent analysis.

The dataset was imported into Python using the Pandas library:

```
1 import pandas as pd
2
3 df = pd.read_csv('data/bike_sharing_dataset.csv')
```

Listing 1: Dataset import using Pandas.

The first observations were inspected using:

```
1 df.head()
```

Listing 2: Inspection of the first observations.

Table 2.2: First five observations of the original dataset.

instant	dteday	season	yr	mnth	hr	holiday	weekday	weekend	workingday	weathersit	temp	atemp	hum	windspeed	windspeed_mph	comfindex	casual	registered	cnt
1	2011-01-01	1	0	1	0	0	6	1	0	1	0.240000	0.287900	0.810000	0.000000	0.000000	0.206000	3	13	16.000000
2	2011-01-01	1	0	1	1	0	6	1	0	1	0.220000	0.272700	0.800000	0.000000	0.000000	0.192000	8	32	40.000000
3	2011-01-01	1	0	1	2	0	6	1	0	1	0.220000	0.272700	0.800000	0.000000	0.000000	0.192000	5	27	32.000000
4	2011-01-01	1	0	1	3	0	6	1	0	1	0.240000	0.287900	0.750000	0.000000	0.000000	0.206000	3	10	13.000000
5	2011-01-01	1	0	1	4	0	6	1	0	1	0.240000	0.287900	0.750000	0.000000	0.000000	0.206000	0	1	1.000000

The dataset dimensions were obtained through:

```
df.shape
```

Listing 3: Dataset dimensions.

while the structure of the variables and their data types were analysed using:

```
df.info()
```

Listing 4: Dataset structure.

Finally, descriptive statistical indices were computed through:

```
df.describe()
```

Listing 5: Descriptive statistics.

Table 2.3: Description of the original dataset.

	instant	season	yr	mnth	hr	holiday	weekday	weekend	workingday	weathersit	temp	atemp	hum	windspeed	windspeed_mph	comfindex	casual	registered	cnt
17379.000000	17379.000000	17379.000000	17379.000000	17379.000000	17379.000000	17379.000000	17379.000000	17379.000000	17379.000000	17379.000000	17379.000000	16708.000000	17379.000000	17379.000000	17379.000000	17379.000000	17379.000000	17279.000000
8690.000000	2.497037	0.502561	6.537775	11.546752	0.028770	3.003683	0.288509	0.682721	1.425283	1.662256	0.475775	0.628924	0.187994	12.736474	0.353215	35.676218	153.786869	189.429018
5017.029500	1.108993	0.500008	3.438776	6.914405	0.167165	2.005771	0.453082	0.465431	0.639357	5.914283	0.171850	0.193458	0.131869	8.196771	0.150330	49.305030	151.357286	181.419685
1.000000	0.000000	0.000000	1.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	1.000000	0.020000	0.000000	0.000000	-1.000000	0.000000	-0.080000	0.000000	0.000000	1.000000
4345.500000	2.000000	0.000000	4.000000	6.000000	0.000000	1.000000	0.000000	0.000000	1.000000	0.340000	0.333300	0.480000	0.104500	7.002000	0.231000	4.000000	34.000000	40.000000
8690.000000	3.000000	1.000000	7.000000	12.000000	0.000000	3.000000	0.000000	1.000000	1.000000	0.500000	0.484800	0.630000	0.194000	12.998000	0.293700	17.000000	115.000000	142.000000
13034.500000	3.000000	1.000000	10.000000	18.000000	0.000000	5.000000	1.000000	1.000000	2.000000	0.660000	0.621200	0.790000	0.253700	16.998000	0.375000	48.000000	220.000000	281.000000
17379.000000	4.000000	1.000000	12.000000	23.000000	1.000000	6.000000	1.000000	1.000000	4.000000	41.000000	1.000000	1.000000	0.850700	56.997000	0.753000	367.000000	886.000000	977.000000

The command `df.shape` returns the number of observations and attributes contained in the dataset. The method `df.info()` provides information on column names, data types and non-null observations, whereas `df.describe()` returns the main descriptive statistical indices for numerical variables.

In particular, the descriptive analysis includes the sample mean, standard deviation, minimum value, maximum value and quartiles. These indices provide an initial indication of the distribution and admissible range of each numerical attribute.

3 Data Preprocessing

3.1 Missing Values

Missing values represent one of the main data quality problems in Machine Learning applications. They may arise because the information was not collected, was incorrectly recorded or was not available for a specific observation.

Different strategies can be adopted to handle missing values. The corresponding observations can be removed, provided that the dataset is sufficiently large and that the removal does not introduce a systematic bias. Alternatively, the missing values can be estimated using statistical measures such as the mean, median or mode, a constant value, or information contained in the other attributes.

The appropriate strategy depends on the meaning of the variable and on the structure of the missing observations. Therefore, a single imputation method should not necessarily be applied to all attributes.

3.1.1 Identification of Missing Values

Pandas represents missing numerical values through the `NaN` value. The number of missing observations for each attribute was computed using:

```
df.isnull().sum()
```

Listing 6: Number of missing values for each attribute.

The total number of missing values was obtained through:

```
df.isnull().sum().sum()
```

Listing 7: Total number of missing values.

The analysis identified missing values only in two attributes:

- `hum`: 671 missing values;
- `cnt`: 100 missing values.

Therefore, the original dataset contained a total of

$$671 + 100 = 771 \tag{3.1}$$

missing values.

3.1.2 Reconstruction of the Target Variable

The target variable `cnt` represents the total number of rented bicycles and is defined as the sum of casual and registered users:

$$\texttt{cnt} = \texttt{casual} + \texttt{registered}. \tag{3.2}$$

Since both components were available for all observations, the missing target values could be reconstructed exactly rather than estimated statistically.

The following operation was applied only to the rows in which `cnt` was missing:

```
missing_cnt = df['cnt'].isnull()

df.loc[missing_cnt, 'cnt'] = ( df.loc[missing_cnt, 'casual'] + df.loc[missing_cnt, '
    registered'])
```

Listing 8: Reconstruction of missing values in the target variable.

3.1.3 Imputation of Relative Humidity

Unlike `cnt`, the missing values of `hum` could not be reconstructed through an exact algebraic relationship. A statistical imputation procedure was therefore required.

A global mean imputation would assign the same humidity value to all missing observations, ignoring the temporal and meteorological structure of the dataset. Since humidity depends on seasonal and weather conditions, the missing values were estimated using the average humidity of observations belonging to comparable temporal and meteorological groups.

```
df['hum'] = df['hum'].fillna( df.groupby(['mnth', 'hr', 'weathersit'])['hum'].transform('
    mean'))
```

Listing 9: Grouped imputation of missing humidity values.

If a group did not contain any valid humidity observation, the remaining missing values were replaced using the overall mean of the attribute as a fallback strategy.

3.1.4 Final Verification

After applying the reconstruction and imputation procedures, the dataset was checked again:

```
df.isnull().sum()
```

Listing 10: Verification after missing-value treatment.

The final check confirmed that no missing values remained in the dataset.

3.2 Anomaly Detection and Correction

Data quality may also be affected by noise, outliers and incorrectly recorded values. An outlier is an observation whose characteristics are substantially different from those of the remaining objects in the dataset. However, an unusual value is not necessarily erroneous, since it may represent a rare but valid observation.

For this reason, anomaly detection was not based exclusively on statistical distance from the mean. Each suspicious value was compared with the expected domain of the corresponding attribute and, whenever possible, with redundant information contained in the dataset.

The descriptive statistics highlighted the following potentially anomalous conditions:

- the categorical variable `season` contained the value 0, although its admissible values range from 1 to 4;
- the normalized variable `temp` contained values greater than 1, with a maximum equal to 41;
- the normalized variable `windspeed` contained negative values;

- the binary variables `workingday`, `holiday` and `weekend` required a logical consistency check;
- the variable `comfindex` contained negative values, which were investigated before deciding whether a correction was necessary.

3.2.1 Season

The attribute `season` is categorical and admits the values

$$\texttt{season} \in \{1, 2, 3, 4\}. \tag{3.3}$$

Therefore, observations with `season`=0 were considered invalid. Since the dataset contains several hourly records for the same date, the correct season could be reconstructed using the non-anomalous observations belonging to that day.

The anomalous rows were first identified using:

```
df[df["season"] == 0]
```

Listing 11: Identification of invalid season values.

A dictionary associating each date with its valid season was then constructed:

```
season_by_day = (    df[df["season"] != 0] .groupby("dteday")["season"] .first().to_dict()
    )
```

Listing 12: Association between date and valid season.

Finally, the invalid values were replaced as follows:

```
df["season"] = df.apply( lambda row: season_by_day[row["dteday"]]    if row["season"] ==
    0 else row["season"],axis=1)
```

Listing 13: Correction of invalid season values.

This procedure exploits the hourly structure of the dataset: all observations recorded on the same date necessarily belong to the same season. Therefore, the correction is based on internal consistency rather than on a statistical estimate.

The absence of remaining invalid values was verified through:

```
df[df["season"] == 0]
```

Listing 14: Verification of season correction.

3.2.2 Temperature

According to the dataset definition, `temp` is a normalized temperature variable and should satisfy

$$0 \leq \texttt{temp} \leq 1. \tag{3.4}$$

However, the preliminary statistics showed a maximum value equal to 41. The anomalous observations were inspected together with the normalized apparent temperature `atemp`:

```
df[df["temp"] > 1][["temp", "atemp"]].describe()
```

Listing 15: Inspection of anomalous temperature values.

The anomalous values ranged approximately between 21 and 41, suggesting that a subset of the observations had been stored in degrees Celsius rather than in normalized form. The original bike-sharing dataset defines normalized temperature as the measured temperature divided by the maximum reference value of 41 °C. Therefore, only the observations satisfying `temp`>1 were rescaled:

$$\texttt{temp}_{\text{corrected}} = \frac{\texttt{temp}_{\text{recorded}}}{41}. \tag{3.5}$$

The correction was implemented through:

```
df.loc[df["temp"] > 1, "temp"] = (
    df.loc[df["temp"] > 1, "temp"] / 41
)
```

Listing 16: Correction of temperature values stored in degrees Celsius.

After the correction, all temperature values belonged to the expected interval:

```
df[df["temp"] > 1].shape[0]
df["temp"].describe()
```

Listing 17: Verification of temperature correction.

The final maximum value was equal to 1, confirming the consistency of the corrected attribute.

3.2.3 Wind Speed

The normalized wind-speed attribute should satisfy

$$0 \leq \texttt{windspeed} \leq 1. \tag{3.6}$$

Nevertheless, some observations contained negative normalized values, which are physically inadmissible. The dataset also provides the corresponding wind speed in miles per hour through the attribute `windspeed_mph`.

The anomalous observations were identified using:

```
df[df["windspeed"] < 0]
```

Listing 18: Identification of negative wind-speed values.

The normalized value was reconstructed from the corresponding physical measurement according to

$$\texttt{windspeed} = \frac{\texttt{windspeed\_mph}}{67}, \tag{3.7}$$

where 67 mph is the reference value used for normalization.

```
df.loc[df["windspeed"] < 0, "windspeed"] = (df.loc[df["windspeed"] < 0, "windspeed_mph"]
    / 67)
```

Listing 19: Correction of negative normalized wind-speed values.

The correction was verified through:

```
df[df["windspeed"] < 0].shape[0]
```

Listing 20: Verification of wind-speed correction.

No negative value remained after the correction.

3.2.4 Logical Consistency of Calendar Attributes

The attributes `workingday`, `holiday` and `weekend` are logically dependent. A day can be classified as a working day only if it is neither a holiday nor part of the weekend.

The expected relationship is

$$\texttt{workingday} = \begin{cases} 1, & \text{if } \texttt{holiday} = 0 \text{ and } \texttt{weekend} = 0, \\ 0, & \text{otherwise.} \end{cases} \tag{3.8}$$

The following conditions were therefore considered inconsistent:

- `workingday`=1 together with `holiday`=1 or `weekend`=1;
- `workingday`=0 together with `holiday`=0 and `weekend`=0.

The check returned zero inconsistent observations. Consequently, no correction of the three calendar attributes was required.

3.2.5 Comfort Index

The descriptive analysis also revealed negative values of `comfindex`, with a minimum equal to -0.08.

These values were not automatically classified as errors. The comfort index is a derived quantity whose sign may carry physical meaning, since negative values can represent uncomfortable environmental conditions. Moreover, the values were contained within a limited and continuous range and did not show an isolated coding pattern comparable to the anomalies detected in `temp` and `windspeed`.

Therefore, the negative values were retained during data cleaning. The attribute was subsequently excluded during feature selection because it was redundant with the meteorological variables from which it was derived.

Table 3.1: Summary of the detected anomalies and adopted corrections.

Attribute	Detected condition	Interpretation	Treatment
`season`	Value equal to 0	Invalid category	Reconstructed by date
`temp`	Values greater than 1	Celsius values in normalized column	Division by 41
`windspeed`	Negative values	Invalid normalized values	Reconstructed from mph
Calendar attributes	Logical inconsistencies	None detected	No correction
`comfindex`	Negative values	Plausible derived values	Retained

3.3 Feature Selection

Feature selection is a dimensionality-reduction procedure aimed at retaining only the attributes that provide useful and non-redundant information for the learning task.

According to the data management methodology, attribute selection can improve the quality and efficiency of the analysis by:

- removing irrelevant attributes;
- eliminating redundant information;
- reducing noise in the dataset;

- decreasing computational time and memory requirements;
- simplifying the interpretation of the resulting model.

A redundant attribute largely duplicates information already contained in another variable, while an irrelevant attribute does not contribute meaningfully to the considered prediction task.

In supervised learning, feature selection must also prevent *data leakage*. Data leakage occurs when the input variables contain information that would not be legitimately available at prediction time or that directly reveals the target value. In such a situation, the estimated model performance would be unrealistically optimistic.

```
columns_drop = [
    "casual",
   "registered",
    "instant",
    "dteday",
    "windspeed_mph",
    "temp",
    "comfindex",
    "weekend",
    "workingday"
]

df_model = df.drop(columns=columns_drop)

df_model.columns
```

Listing 21: Removal of redundant, irrelevant and leakage-related attributes.

3.3.1 Data Leakage Variables

The attributes `casual` and `registered` were removed because their sum exactly defines the target variable:

$$\texttt{cnt} = \texttt{casual} + \texttt{registered}. \tag{3.9}$$

Including either of these variables among the predictors would introduce data leakage, since they contain direct information about the target value. In a real forecasting application, the number of casual and registered rentals would not be known before predicting the total hourly demand.

Their inclusion would therefore lead to an unrealistically high predictive performance and would not represent a valid forecasting problem.

3.3.2 Record Identifier

The attribute `instant` is a progressive identifier assigned to each observation. It does not describe any physical, temporal or meteorological characteristic of the bike-sharing system.

Although its numerical value increases with time, it is not an explanatory variable and may introduce an artificial ordering into the model. It was therefore classified as an irrelevant attribute and removed.

3.3.3 Calendar Date

The attribute `dteday` contains the complete calendar date. Its temporal information is already represented through the attributes `yr`, `mnth`, `weekday` and `hr`.

Moreover, a raw date stored as a string cannot be directly used by the considered models without an appropriate transformation. Since the relevant components of the date were already available as separate attributes, `dteday` was considered redundant and removed.

3.3.4 Temperature Variables

The dataset contains both the normalized actual temperature, `temp`, and the normalized apparent temperature, `atemp`.

These two variables describe closely related environmental conditions and are expected to be strongly correlated. Retaining both would introduce redundant information.

The attribute `atemp` was retained because the perceived temperature may be more directly related to the user's decision to rent a bicycle, whereas `temp` was removed.

3.3.5 Wind-Speed Representation

The attributes `windspeed` and `windspeed_mph` represent the same physical quantity using two different scales.

Their relationship is

$$\texttt{windspeed} = \frac{\texttt{windspeed\_mph}}{67}. \tag{3.10}$$

Therefore, the two variables contain equivalent information. The normalized attribute `windspeed` was retained, while `windspeed_mph` was removed to avoid exact redundancy.

3.3.6 Derived Comfort Index

The attribute `comfindex` is a derived outdoor comfort indicator. Since it is computed from meteorological quantities already represented in the dataset, such as temperature, humidity and wind speed, it does not constitute an independent measurement.

Retaining it together with its underlying meteorological variables would duplicate part of the same information. For this reason, `comfindex` was classified as redundant and removed.

3.3.7 Redundant Calendar Indicators

The binary attributes `weekend` and `workingday` were removed because their information can be reconstructed from the retained calendar variables.

The weekend condition is determined by the value of `weekday`, whereas the working-day condition depends on both `weekday` and `holiday`:

$$\texttt{weekend} = \begin{cases} 1, & \text{if the day is Saturday or Sunday,} \\ 0, & \text{otherwise,} \end{cases} \tag{3.11}$$

and

$$\texttt{workingday} = \begin{cases} 1, & \text{if the day is neither a weekend nor a holiday,} \\ 0, & \text{otherwise.} \end{cases} \tag{3.12}$$

Consequently, both attributes are deterministic functions of variables already retained in the dataset. They were therefore removed to reduce redundancy.

3.3.8 Retained Attributes

After feature selection, the working dataset contained the following variables:

- `yr`;
- `mnth`;
- `hr`;
- `holiday`;
- `weekday`;
- `season`;
- `weathersit`;
- `atemp`;
- `hum`;
- `windspeed`;
- `cnt`.

The target variable `cnt` was retained in the working DataFrame but was subsequently separated from the explanatory variables before model training.

Table 3.2: Summary of the removed attributes and corresponding motivations.

Attribute	**Reason for removal**	**Motivation**
`casual`, `registered`	Data leakage	Their sum exactly defines the target variable `cnt`.
`instant`	Irrelevant	Progressive record identifier with no physical meaning.
`dteday`	Redundant	Temporal information already represented by year, month, weekday and hour.
`temp`	Redundant	Strongly related to the retained apparent temperature `atemp`.
`windspeed_mph`	Redundant	Same physical quantity as normalized `windspeed`.
`comfindex`	Redundant	Derived from meteorological information already available.
`weekend`	Redundant	Can be reconstructed from `weekday`.
`workingday`	Redundant	Can be reconstructed from `weekday` and `holiday`.

3.4 Feature Engineering

Feature engineering consists of transforming the available attributes or constructing new ones in order to obtain a representation that is more suitable for the learning algorithms.

In the considered dataset, the temporal attributes `hr`, `mnth` and `weekday` are represented by integer values but describe periodic phenomena. Moreover, `season` and `weathersit` are categorical attributes whose numerical codes do not represent actual numerical quantities.

The feature-engineering procedure therefore included:

- the transformation of periodic temporal attributes;
- the encoding of categorical attributes;
- the removal of the original representations when their information was already contained in the newly generated features.

3.4.1 Transformation of Temporal Attributes

The attributes `hr`, `mnth` and `weekday` describe periodic quantities. For example, hour 23 and hour 0 are consecutive, although their original numerical values are distant.

Each periodic attribute x, characterized by a period T, was represented through two new features:

$$x_{\sin} = \sin\left(\frac{2\pi x}{T}\right), \tag{3.13}$$

$$x_{\cos} = \cos\left(\frac{2\pi x}{T}\right). \tag{3.14}$$

This transformation preserves the periodic structure of the original attribute. After constructing the new temporal features, the original attributes `hr`, `mnth` and `weekday` were removed in order to avoid a redundant representation of the same information.

3.4.2 Encoding of Categorical Attributes

The attributes `season` and `weathersit` are categorical.

Their integer codes identify different categories but do not represent numerical magnitudes. Therefore, they were transformed into binary attributes using one-hot encoding.

For each possible category, a new binary variable was introduced:

$$d_k = \begin{cases} 1, & \text{if the observation belongs to category } k, \\ 0, & \text{otherwise.} \end{cases} \tag{3.15}$$

- `season_1`, `season_2`, `season_3`, `season_4`;
- `weathersit_1`, `weathersit_2`, `weathersit_3`, `weathersit_4`.

3.4.3 Reference Categories

One binary attribute was removed from each group of encoded categories, since the complete set would provide a redundant representation of the original categorical variable.

The first category of each attribute was selected as the reference category:

```
df_model = df_model.drop(
    columns=["season_1", "weathersit_1"]
)
```

Listing 22: Removal of the reference categories.

The effects associated with the remaining categories are therefore evaluated with respect to `season_1` and `weathersit_1`.

Table 3.3: Feature-engineering operations.

Attribute	Transformation	Generated attributes
`hr`	Periodic transformation	`hr_sin`, `hr_cos`
`mnth`	Periodic transformation	`mnth_sin`, `mnth_cos`
`weekday`	Periodic transformation	`weekday_sin`, `weekday_cos`
`season`	Categorical encoding	`season_2`, `season_3`, `season_4`
`weathersit`	Categorical encoding	`weathersit_2`, `weathersit_3`, `weathersit_4`

3.5 Exploratory Data Analysis

Exploratory Data Analysis was performed to identify the main temporal and meteorological patterns contained in the dataset.

3.5.1 Daily Demand

The hourly rentals were aggregated by date in order to obtain the total daily demand.

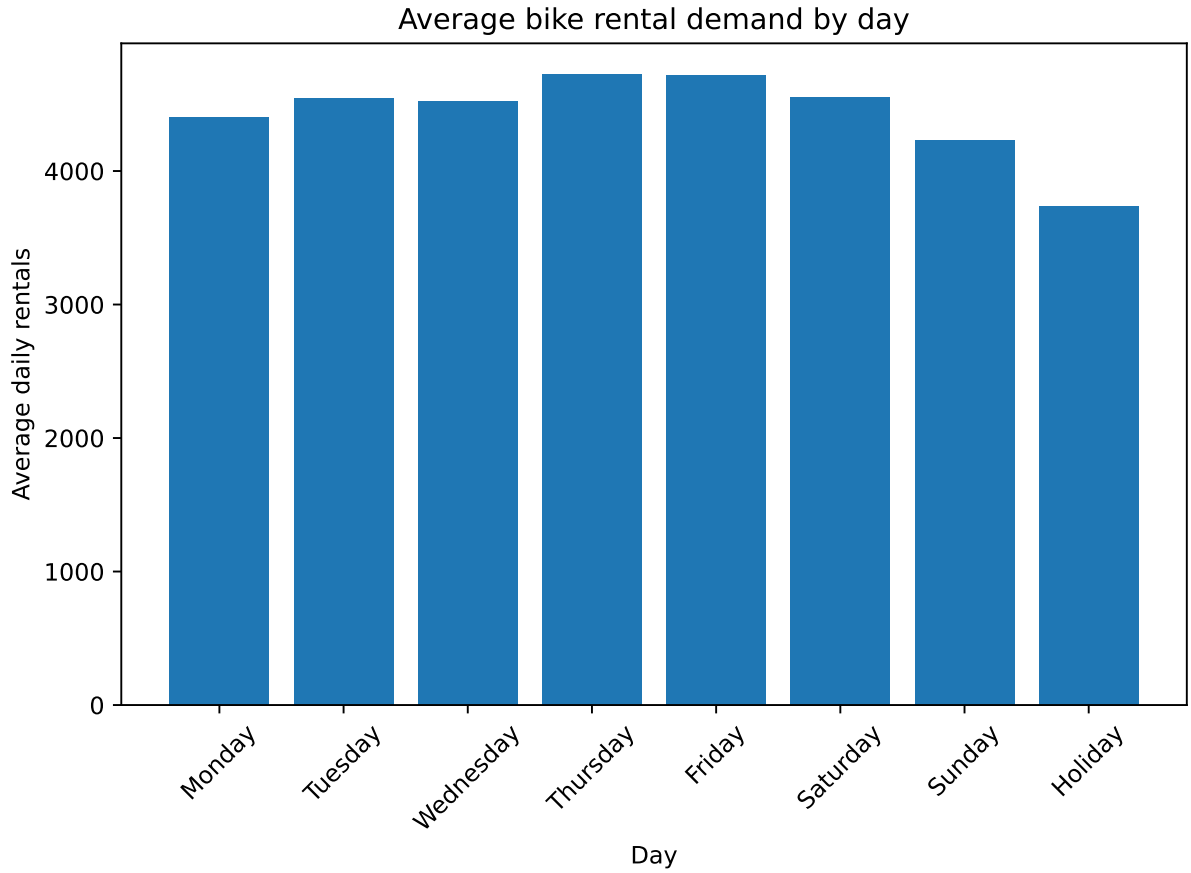


Figure 3.1: Evolution of the total daily bike-sharing demand.

Figure 3.1 shows a strong variability of the daily demand over time.

3.5.2 Hourly Demand

The average demand was computed for each hour of the day.

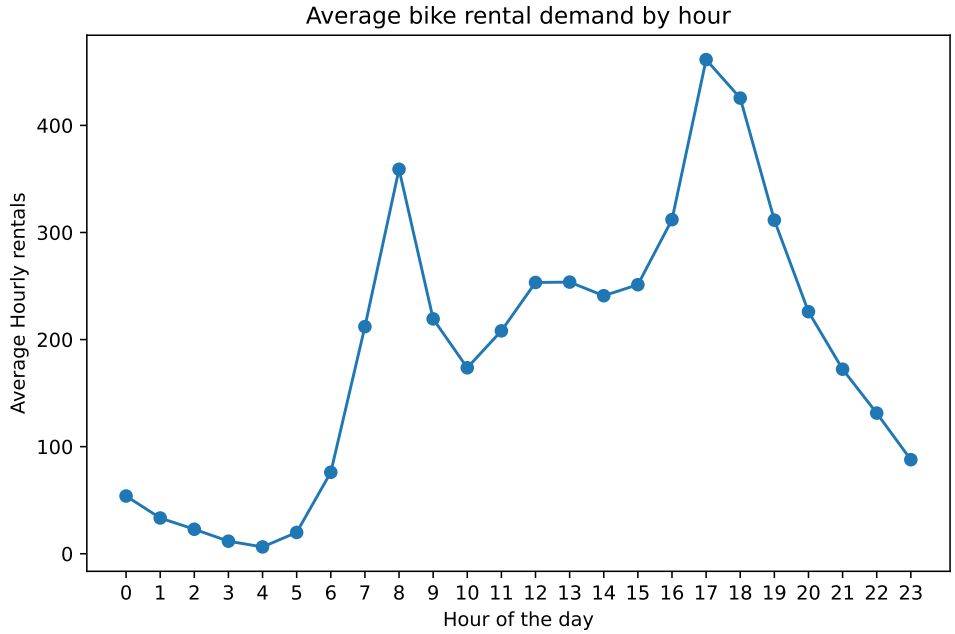


Figure 3.2: Average bike-sharing demand for each hour of the day.

Figure 3.2 highlights a clear daily pattern, with lower demand during the night and higher demand during daytime hours.

3.5.3 Monthly Demand

The hourly rentals were aggregated by month and averaged over the two considered years.

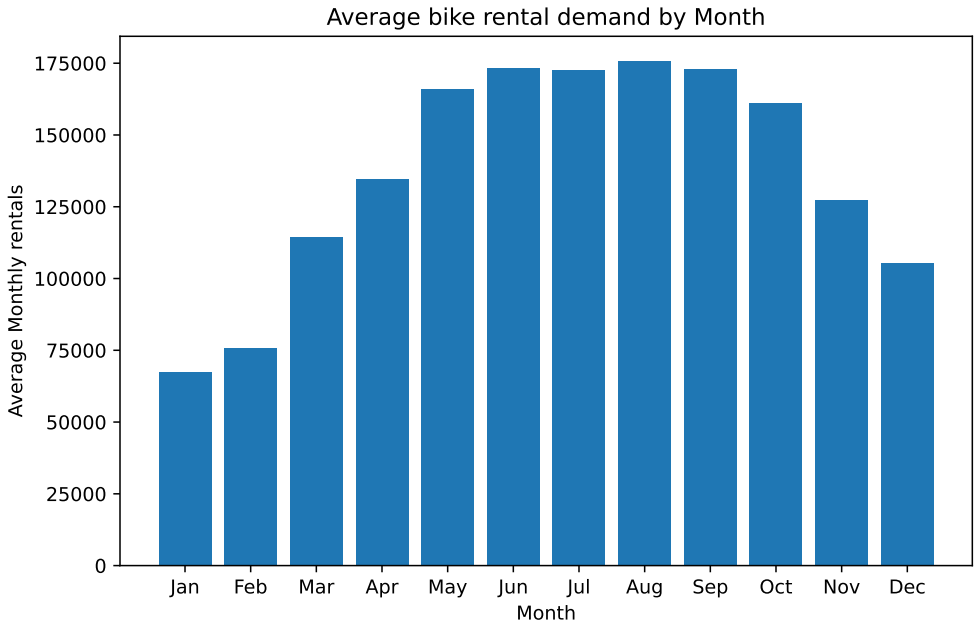
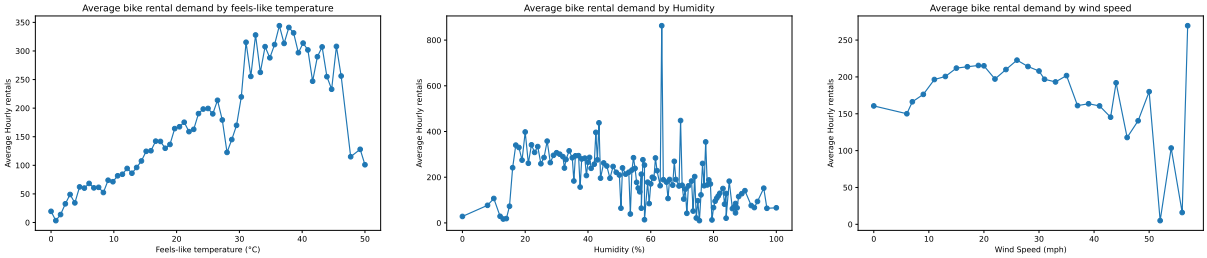


Figure 3.3: Average monthly bike-sharing demand.

Figure 3.3 shows that demand changes across the year, indicating a seasonal behaviour.

3.5.4 Meteorological Attributes

The average demand was analysed as a function of feels-like temperature, humidity and wind speed.



(a) Feels-like temperature. (b) Humidity. (c) Wind speed.

Figure 3.4: Average bike-sharing demand as a function of the retained meteorological attributes.

Figure 3.4 indicates that bike-sharing demand is influenced by environmental conditions. In particular, the demand generally increases with feels-like temperature, while high humidity is mainly associated with lower demand values. The effect of wind speed appears less pronounced.

4 Model Development and Evaluation

The prediction of `cnt` was formulated as a supervised regression problem. A Linear Regression model was first used as a baseline, followed by a Random Forest Regressor and a tuning phase based on Grid Search.

4.1 Data Splitting

The explanatory variables and the target were separated as follows:

```
X = df_model.drop("cnt", axis=1)
y = df_model["cnt"]
```

Listing 23: Separation of input attributes and target variable.

The dataset was divided into training and test sets:

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)
```

Listing 24: Training and test split.

The resulting training set contained 13 903 observations, while the test set contained 3 476 observations.

4.2 Performance Measures

The models were evaluated using the Root Mean Squared Error, the Mean Absolute Error and the coefficient of determination.

$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}, \tag{4.1}$$

$$\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|, \tag{4.2}$$

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}. \tag{4.3}$$

Lower RMSE and MAE values indicate smaller prediction errors, while larger values of R^2 indicate a better representation of the target variability.

4.3 Baseline Model: Linear Regression

Linear Regression was adopted as the baseline model.

```
from sklearn.linear_model import LinearRegression

linear_model = LinearRegression()
linear_model.fit(X_train, y_train)

y_pred_linear = linear_model.predict(X_test)
```

Listing 25: Training and prediction with Linear Regression.

The model assumes a linear relationship between the predictors and the target:

$$\hat{y} = \beta_0 + \sum_{j=1}^{p} \beta_j x_j. \tag{4.4}$$

The obtained results are reported in Table 4.1.

Table 4.1: Performance of the Linear Regression model.

Metric	**Value**
RMSE	124.78505
MAE	91.48109
R^2	0.50826

The relatively large errors and the R^2 value close to 0.5 indicate that a linear model is not sufficient to represent the complete demand behaviour.

4.4 Advanced Model: Random Forest

A Random Forest Regressor was then trained:

```
from sklearn.ensemble import RandomForestRegressor

random_forest_model = RandomForestRegressor(
    n_estimators=50,
    random_state=42,
    n_jobs=-1
)

random_forest_model.fit(X_train, y_train)

y_pred_rf = random_forest_model.predict(X_test)
```

Listing 26: Training and prediction with Random Forest.

Random Forest combines the predictions of several decision trees. In regression, the final prediction is obtained by averaging the outputs of the individual trees:

$$\hat{y} = \frac{1}{B} \sum_{b=1}^{B} \hat{y}_b, \tag{4.5}$$

where B is the number of trees.
The obtained results are shown in Table 4.2.

Table 4.2: Performance of the Random Forest Regressor.

Metric	**Value**
RMSE	44.10258
MAE	26.75117
R^2	0.93858

The Random Forest strongly improved the prediction accuracy compared with the Linear Regression baseline, indicating that the relationship between demand and the input attributes is not purely linear.

4.5 Hyperparameter Tuning

The Random Forest hyperparameters were tuned using Grid Search with three-fold cross-validation.

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    "n_estimators": [100, 200],
    "max_depth": [None, 10, 20],
    "min_samples_split": [2, 5],
    "min_samples_leaf": [1, 2]
}

rf_for_grid = RandomForestRegressor(
    random_state=42,
    n_jobs=-1
)

gs = GridSearchCV(
    estimator=rf_for_grid,
    param_grid=param_grid,
    scoring="neg_root_mean_squared_error",
    cv=3,
    n_jobs=-1,
    verbose=2
)

gs.fit(X_train, y_train)
```

Listing 27: Random Forest hyperparameter tuning.

The best hyperparameter configuration was:

- `n_estimators` = 200;
- `max_depth` = None;
- `min_samples_split` = 2;
- `min_samples_leaf` = 1.

The best cross-validation RMSE was:

$$\text{RMSE}_{CV} = 48.73327. \tag{4.6}$$

The tuned model produced the following test results:

Table 4.3: Performance of the tuned Random Forest model.

Metric	Value
RMSE	43.77932
MAE	26.34053
R^2	0.93947

4.6 Model Comparison

Table 4.4: Comparison of the considered regression models.

Model	**RMSE**	**MAE**	R^2
Linear Regression	124.78505	91.48109	0.50826
Random Forest	44.10258	26.75117	0.93858
Tuned Random Forest	43.77932	26.34053	0.93947

The Random Forest models clearly outperformed the Linear Regression baseline. Hyperparameter tuning produced only a limited improvement, since the initial Random Forest configuration already provided good predictive performance.

Among the tested models, the tuned Random Forest achieved the lowest RMSE and MAE and the highest value of R^2.

5 Prediction and Interpretation

The tuned Random Forest model was used to predict the hourly number of rentals contained in the test set.

5.1 Predicted Values

Table 5.1 reports some examples of actual and predicted demand values.

Table 5.1: Examples of actual and predicted hourly rentals.

Actual	Predicted
425.000000	376.300000
88.000000	114.590000
4.000000	12.385000
526.000000	500.550000
13.000000	13.485000

The predictions are generally close to the corresponding observed values. Larger differences are mainly found for some observations characterized by high or unusual demand.

5.2 Actual and Predicted Demand

The relationship between actual and predicted values is shown in Figure 5.1.

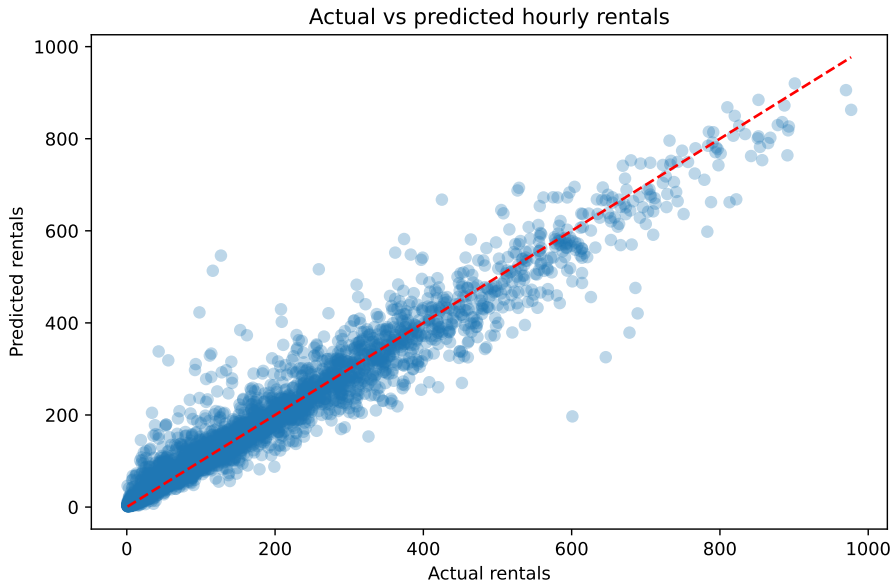


Figure 5.1: Actual and predicted hourly bike rentals obtained with the tuned Random Forest.

The dashed line represents the ideal condition

$$\hat{y} = y, \tag{5.1}$$

for which the predicted value is equal to the observed value.

Most observations are distributed close to the reference line, confirming the good predictive performance of the tuned Random Forest. A larger dispersion is visible for some intermediate and high-demand observations.

5.3 Feature Importance

The Random Forest model provides an importance value for each input feature. The most important variables are reported in Table 5.2.

Table 5.2: Most important features according to the tuned Random Forest.

Feature	Importance
hr_cos	0.297708
hr_sin	0.245676
atemp	0.137347
yr	0.087974
weekday_cos	0.064359

The cyclic hour features, `hr_cos` and `hr_sin`, have the largest importance values. This confirms that the hour of the day is the main factor affecting bike-sharing demand.

The apparent temperature `atemp` is the most relevant meteorological feature. The variable `yr` also has a significant contribution, indicating a difference in demand between the two considered years.

Overall, the feature-importance analysis confirms that hourly patterns, temperature and long-term temporal changes are the main factors used by the model to predict bike-sharing demand.

5.4 Conclusions and Business Recommendations

The analysis showed that the tuned Random Forest was the best-performing model for predicting hourly bike-sharing demand.

Compared with the Linear Regression baseline, the tuned Random Forest achieved substantially lower prediction errors and a higher coefficient of determination. The final model obtained an RMSE of 43.77932, an MAE of 26.34053 and an R^2 value of 0.93947.

The comparison between actual and predicted values showed that most observations were located close to the ideal prediction line. Larger errors remained mainly for some high-demand observations.

The feature-importance analysis indicated that the hour of the day was the most influential factor, followed by apparent temperature and the year variable. Therefore, bike-sharing demand is mainly affected by daily usage patterns, environmental conditions and the general growth of the service over time.

From a business perspective, the model can support the management of the bike-sharing fleet in several ways:

- bicycles should be redistributed before the main demand peaks identified during the day;

- larger bike availability should be planned during periods characterized by favourable temperatures;
- lower demand should be expected during periods of high humidity or adverse weather conditions;
- maintenance and battery-charging operations should preferably be scheduled during low-demand hours;
- the forecasting model should be periodically retrained using recent data in order to account for changes in user behaviour and growth of the service.

Overall, the developed model provides a useful decision-support tool for improving bicycle allocation, reducing the risk of empty or saturated stations and increasing the efficiency of fleet operations.

6 E-Bike Charging Hub Placement

The Operations Research task concerns the placement of charging hubs within a network of 20 bike-sharing stations.

The input data contain a symmetric distance matrix and a coverage radius equal to

$$D = 3.5 \text{ km}. \tag{6.1}$$

A station is considered covered when at least one selected charging hub is located within the coverage radius.

6.1 Mathematical Formulation

Let

$$\mathcal{N} = \{1, \ldots, 20\} \tag{6.2}$$

be the set of stations, and let d_{ij} denote the distance between stations i and j. For each station j, the coverage set is defined as

$$C(j) = \{i \in \mathcal{N} \mid d_{ij} \leq D\}. \tag{6.3}$$

The binary decision variable is

$$Y_i = \begin{cases} 1, & \text{if station } i \text{ is selected as a charging hub,} \\ 0, & \text{otherwise.} \end{cases} \tag{6.4}$$

The objective is to minimize the total number of selected charging hubs:

$$\min \sum_{i \in \mathcal{N}} Y_i. \tag{6.5}$$

Each station must be covered by at least one selected hub:

$$\sum_{i \in C(j)} Y_i \geq 1, \qquad \forall j \in \mathcal{N}. \tag{6.6}$$

Finally,

$$Y_i \in \{0, 1\}, \qquad \forall i \in \mathcal{N}. \tag{6.7}$$

6.2 PuLP Implementation

The data were imported from the provided JSON file:

```
import json

with open("data/or_data.json", "r") as f:
    or_data = json.load(f)

stations = or_data["stations"]
distance_matrix = or_data["distance_matrix"]
coverage_radius = or_data["coverage_radius"]
```

Listing 28: Import of the Operations Research data.

For each station j, the set of possible hubs capable of covering it was constructed as follows:

```
coverage_sets = {}

for j in range(len(stations)):
    coverage_sets[j] = []

    for i in range(len(stations)):
        if distance_matrix[i][j] <= coverage_radius:
            coverage_sets[j].append(i)
```

Listing 29: Construction of the coverage sets.

The binary optimization model was then implemented using PuLP:

```
import pulp

model = pulp.LpProblem(
    "E_Bike_Charging_Hub_Placement",
    pulp.LpMinimize
)

Y = pulp.LpVariable.dicts(
    "Hub",
    range(len(stations)),
    cat="Binary"
)

model += pulp.lpSum(
    Y[i] for i in range(len(stations))
)

for j in range(len(stations)):
    model += (
        pulp.lpSum(
            Y[i] for i in coverage_sets[j]
        ) >= 1
    )

model.solve()
```

Listing 30: Set-covering model implemented with PuLP.

The solver returned an optimal solution.

6.3 Optimal Hub Placement

The minimum number of charging hubs required to cover all stations was

$$p^* = 4. \tag{6.8}$$

The selected charging-hub locations were:

- `Station_4`;
- `Station_11`;
- `Station_12`;
- `Station_19`.

The stations covered by each selected hub are reported in Table 6.1.

Table 6.1: Optimal charging hubs and corresponding covered stations.

Charging hub	**Covered stations**
Station_4	Station_4, Station_7, Station_20
Station_11	Station_1, Station_6, Station_8, Station_11, Station_14, Station_18
Station_12	Station_2, Station_5, Station_7, Station_9, Station_12, Station_16, Station_17, Station_18
Station_19	Station_3, Station_10, Station_13, Station_15, Station_19, Station_20

The four selected hubs jointly provide complete coverage of the network. Some stations are covered by more than one hub, while every station is located within the coverage radius of at least one selected location.

6.4 Sensitivity Analysis

A sensitivity analysis was performed by reducing the available number of charging hubs from p^* to

$$p^* - 1 = 3. \tag{6.9}$$

With only three hubs, complete coverage is no longer possible. Therefore, a new binary variable was introduced:

$$U_j = \begin{cases} 1, & \text{if station } j \text{ is uncovered,} \\ 0, & \text{otherwise.} \end{cases} \tag{6.10}$$

The objective becomes the minimization of the number of uncovered stations:

$$\min \sum_{j \in \mathcal{N}} U_j. \tag{6.11}$$

The number of available hubs is fixed through

$$\sum_{i \in \mathcal{N}} Y_i = p^* - 1. \tag{6.12}$$

The coverage constraints are modified as follows:

$$\sum_{i \in C(j)} Y_i + U_j \geq 1, \qquad \forall j \in \mathcal{N}. \tag{6.13}$$

The optimal solution under the reduced budget selected the following three hubs:

- Station_2;
- Station_7;
- Station_19.

The minimum number of uncovered stations was

$$\sum_{j \in \mathcal{N}} U_j = 1. \tag{6.14}$$

The only station left uncovered was

$$\texttt{Station\_1}. \tag{6.15}$$

Table 6.2: Charging hubs selected with a budget of three hubs.

Charging hub	**Covered stations**
Station_2	Station_2, Station_5, Station_6, Station_8, Station_9, Station_11, Station_12, Station_14, Station_15, Station_16, Station_17, Station_18
Station_7	Station_4, Station_7, Station_20
Station_19	Station_3, Station_10, Station_13, Station_15, Station_19, Station_20

6.5 Operations Research Conclusions

The set-covering model showed that at least four charging hubs are required to guarantee complete coverage of the 20-station network.

When the available budget was reduced to three hubs, the model was able to cover 19 of the 20 stations. Therefore, reducing the infrastructure by one charging hub causes only one station, Station_1, to remain uncovered.

From a planning perspective, four hubs represent the minimum infrastructure required for complete service. If the operator accepts a limited reduction in coverage, three hubs provide a lower-cost alternative while maintaining service for 95% of the stations.